

Single-cell transcriptomic analysis of *Arabidopsis thaliana* root cells under contrasting nitrogen availability

Abstract

Nitrogen is a critical macronutrient governing plant growth, development, and adaptive responses. Here we describe a comprehensive single-cell RNA sequencing (scRNA-seq) pipeline applied to *Arabidopsis thaliana* roots grown under three nitrogen regimes: nitrogen-deficient (0N), low-nitrogen (0.5N), and sufficient-nitrogen (5N) conditions. Five samples — one replicate for 0N and two biological replicates each for 0.5N and 5N — were processed from raw reads through ambient RNA correction (CellBender), quality control, doublet removal, integration (Harmony), clustering, and cell-type annotation. The resulting atlas reveals the transcriptional heterogeneity of root cell populations and identifies gene expression programmes specifically modulated by nitrogen availability. All analyses were performed within a reproducible Docker environment (`mvergara19/scrnaseq_docker:latest`) containing R 4.5 with Seurat 5, monocle3, harmony, and DESeq2.

1 Introduction

Plants continuously sense and respond to fluctuations in nitrogen availability in the rhizosphere. Nitrogen signalling cascades coordinate root architecture, cell-type-specific gene expression, and whole-plant metabolism. Bulk transcriptomic approaches have established key regulatory nodes in this response; however, they mask the cell-type-specific resolution required to understand how individual root cell populations — epidermis, cortex, endodermis, stele, and meristematic zones — each contribute to the systemic nitrogen response.

Single-cell RNA sequencing provides the resolution necessary to dissect these contributions. By profiling the transcriptomes of thousands of individual root protoplasts simultaneously, scRNA-seq enables the unsupervised identification of cell types, the detection of rare populations, and the reconstruction of developmental trajectories, all within a single experiment.

The present study applies the 10x Genomics Chromium platform to profile *Arabidopsis* root cells exposed to three nitrogen concentrations. Key analytical steps include:

1. Ambient RNA removal with CellBender to reduce background contamination from the protoplasting buffer.
2. Per-cell quality control filtering, with thresholds calibrated for the relatively small *Arabidopsis* root cell transcriptome.

3. Doublet detection using DoubletFinder to exclude artefactual cell multiplets.
4. Batch correction using Harmony to integrate five samples (two conditions with biological replicates) into a shared embedding.
5. Leiden clustering with resolution selection guided by clustree and k-means elbow analysis.
6. Cell-type annotation by two complementary approaches: marker gene cross-referencing and label transfer from a published *Arabidopsis* root atlas (GSE273033).

2 Materials

2.1 Experimental design

Sample	Label	Condition	Replicates
Sample_0N	0N	Nitrogen-deficient	1
Sample_05N_R1	0.5N_R1	Low nitrogen	2
Sample_05N_R2	0.5N_R2	Low nitrogen	
Sample_5N_R1	5N_R1	Sufficient nitrogen	2
Sample_5N_R2	5N_R2	Sufficient nitrogen	

2.2 Computational environment

All analyses were executed inside the Docker container `mvergara19/scrnaseq_docker:latest`.
`docker run --rm -it -v $(pwd):/workspace mvergara19/scrnaseq_docker:latest bash`

2.3 Software versions

Tool	Version	Role
R	4.5	Primary analysis environment
Seurat	5.4.0	QC, normalization, clustering
DoubletFinder	2.0.6	Doublet detection
harmony	1.2.4	Batch correction
monocle3	1.4.26	Trajectory inference
DESeq2	1.50.2	Differential expression
clustree	0.5.1	Resolution selection
zellkonverter	1.20.1	h5ad export
CellBender	0.3.0	Ambient RNA removal
CellRanger	9.0.1	Read alignment and UMI counting

2.4 Custom function library

All reusable functions are centralised in `ScRNA_Analysis_Functions.R`. Key functions are described in Table 1. A visualisation helper (`plot_integrated_clusters`) is defined separately in `custom_seurat.R`.

Table 1. Custom R functions used in this analysis.

Function	Role
<code>load_cellbender_filtered_h5()</code>	Reads CellBender HDF5 output into a Seurat object
<code>plot_qc_violin_grid()</code>	4-panel QC violin plot per sample
<code>resumen_nFeature_plot()</code>	Boxplot + quartile/quintile summary tables
<code>process_sample()</code>	Full per-sample pipeline (load, QC filter, doublet removal)
<code>doubletfinder_pipeline()</code>	DoubletFinder with automated parameter sweep
<code>generate_pseudobulk()</code>	Aggregates counts by sample or condition
<code>normalizar_bulk_pseudobulk()</code>	DESeq2-based normalization of pseudobulk vs bulk
<code>clasificar_residuos()</code>	Gene classification by linear model residuals
<code>find_markers()</code>	FindAllMarkers with disk caching
<code>annotate_by_markers()</code>	Cluster annotation from marker gene reference list
<code>annotate_by_reference()</code>	Label transfer from reference Seurat object
<code>exportar_para_scanpy()</code>	Seurat to h5ad export via zellkonverter

3 Methods

3.1 Read alignment and UMI counting

Raw FASTQ files were processed with CellRanger 9.0.1 using the TAIR10 *Arabidopsis thaliana* reference genome. CellRanger performs read alignment (STAR), barcode correction, and UMI deduplication, producing per-sample filtered feature-barcode matrices.

```
cellranger count \
  --id=sample_id \
  --fastqs=/workspace/fastqs \
  --sample=sample_name \
  --transcriptome=/workspace/refdata-TAIR10
```

3.2 Ambient RNA removal

Ambient RNA — transcripts released by lysed protoplasts and captured by empty droplets — was quantified and removed using CellBender 0.3.0. CellBender fits a variational autoencoder that models the expected count distribution under the ambient contamination hypothesis, producing a corrected count matrix:

```
cellbender remove-background \
  --input /workspace/raw_feature_bc_matrix.h5 \
  --output /workspace/cellbender_output.h5
```

The corrected HDF5 files are stored under `cellbender/` and serve as input for all downstream R analyses. A custom reader (`load_cellbender_filtered_h5`) parses the CellBender HDF5 format directly into a Seurat object using the sparse CSR matrix components.

3.3 Project initialisation

The analysis environment is initialised by loading all required libraries and sourcing the custom function files:

```
setwd("/home/mvergara/projects2/eleo/ScRNA/")
source("metodologia/custom_seurat.R")
source("metodologia/ScRNA_Analysis_Functions.R")
```

Sample metadata and colour assignments are defined centrally:

```
samples <- list(
  list(file = "cellbender/Sample_0N_cellbender_filtered.h5",
        label = "0N", condition = "0N"),
  list(file = "cellbender/Sample_0.5N_R1_cellbender_filtered.h5",
        label = "0.5N_R1", condition = "0.5N"),
  list(file = "cellbender/Sample_0.5N_2_cellbender_filtered.h5",
        label = "0.5N_R2", condition = "0.5N"),
  list(file = "cellbender/Sample_5N_R1_cellbender_filtered.h5",
        label = "5N_R1", condition = "5N"),
  list(file = "cellbender/Sample_5N_2_cellbender_filtered.h5",
        label = "5N_R2", condition = "5N")
)

colors <- c("0N" = "#66c2a5", "0.5N_R1" = "#fc8d62",
            "0.5N_R2" = "#fc8d62", "5N_R1" = "#8da0cb",
            "5N_R2" = "#8da0cb")
```

3.4 Quality control

3.4.1 Per-cell QC metrics

For each cell, four QC metrics are computed:

- **nFeature_RNA** — number of distinct genes detected; low values indicate empty droplets or highly damaged cells.
- **nCount_RNA** — total UMI count; extreme values may indicate doublets or poor-quality cells.
- **percent.mt** — percentage of counts mapping to mitochondrial genes; elevated in damaged protoplasts.
- **percent.cp** — percentage of counts mapping to chloroplast/plastid genes; elevated in mesophyll contamination. Only applicable to plant datasets.

The `process_sample()` function accepts `mt_pattern` and `cp_pattern` arguments so that the pipeline can be applied to any organism without modifying the function body. Setting `cp_pattern = NULL` disables chloroplast QC entirely (Table 2).

Table 2. Organellar gene name patterns by organism.

Organism	mt_pattern	cp_pattern
<i>Arabidopsis thaliana</i>	^ATMG	^ATCG
<i>Homo sapiens</i>	^MT-	NULL

Organism	mt_pattern	cp_pattern
<i>Mus musculus</i>	<code>^mt-</code>	NULL
<i>Danio rerio</i> (zebrafish)	<code>^mt-</code>	NULL
<i>Oryza sativa</i> (rice)	<code>^0sMG</code>	<code>^0sCG</code>

```
# Arabidopsis (default)
seurat_list <- lapply(samples, process_sample,
  min_features = 200, max_mt = 5,
  mt_pattern   = "^ATMG",
  cp_pattern   = "^ATCG")
```

```
# Human
seurat_list <- lapply(samples, process_sample,
  min_features = 200, max_mt = 20,
  mt_pattern   = "^MT-",
  cp_pattern   = NULL)
```

```
# Mouse
seurat_list <- lapply(samples, process_sample,
  min_features = 200, max_mt = 20,
  mt_pattern   = "^mt-",
  cp_pattern   = NULL)
```

Note that *Arabidopsis* organellar gene prefixes differ from mammalian conventions: mitochondrial genes carry the prefix ATMG and plastid genes ATCG.

3.4.2 Pre-filter QC visualisation

Four-panel violin plots are generated for each sample before filtering to visualise the raw distributions and guide threshold selection (Figure 1):

```
seurat_list_raw <- lapply(samples, process_sample)
plots_pre <- imap(seurat_list_raw,
  ~ plot_qc_violin_grid(.x, .y, colors[[".y"]]))

ggsave("metodologia/qc_violin_all_samples_prefilter.png",
  wrap_plots(plots_pre, ncol = 1),
  width = 14, height = 6 * length(plots_pre), dpi = 300)
```

3.4.3 Filtering thresholds

Based on pre-filter distributions, the following thresholds were applied uniformly across all samples:

Metric	Threshold
nFeature_RNA	> 200
percent.mt	< 5%
percent.cp	< default (100, not applied)

```
seurat_list <- lapply(samples, process_sample,
                      min_features = 200,
                      max_mt       = 5)
```

Post-filter distributions are re-examined (Figure 2) to confirm that filtering retained the expected cell population while removing outliers.

3.5 Doublet detection

Each sample is independently processed through DoubletFinder before merging. The algorithm simulates artificial doublets by averaging pairs of real cell profiles and identifies real cells whose transcriptomes closely resemble these artificial multiplets.

The optimal neighbourhood size parameter (pK) is identified from a parameter sweep via the BC-metric criterion:

```
doubletfinder_pipeline(obj,
                       etiqueta      = sample_info$label,
                       PCs           = 1:20,
                       resolution    = 0.5,
                       return_singlets = TRUE)
```

The expected doublet rate for 10x Genomics libraries is approximately 0.8% per 1,000 cells recovered and is set proportionally per sample. Only cells classified as Singlet are retained.

3.6 Integration and preprocessing

3.6.1 Merging samples

Filtered, singlet-only Seurat objects are merged into a single object using `purrr::reduce`:

```
pbmc_harmony <- reduce(seurat_list, merge)
```

3.6.2 Normalisation and dimensionality reduction

The merged object undergoes standard Seurat preprocessing: log-normalisation (scale factor 10,000), identification of the 2,000 most variable genes (VST method), data scaling, and PCA (30 PCs):

```
pbmc_harmony <- pbmc_harmony %>%
  NormalizeData(verbose = FALSE) %>%
  FindVariableFeatures(selection.method = "vst", nfeatures = 2000) %>%
  ScaleData(verbose = FALSE) %>%
  RunPCA(npcs = 30, verbose = FALSE) %>%
  RunUMAP(reduction = "pca", dims = 1:30, verbose = FALSE)
```

A pre-integration UMAP coloured by `orig.ident` is saved to assess the extent of batch effects before correction (Figure 3).

3.6.3 Harmony batch correction

Harmony iteratively adjusts the PCA embedding to align sample-specific clusters while preserving biological variance. Integration is performed over `orig.ident`:

```
pbmc_harmony <- pbmc_harmony %>%
  RunHarmony("orig.ident", plot_convergence = FALSE)
```

The corrected embedding is used as input (`reduction = "harmony"`) for all subsequent neighbour graph and UMAP computations.

3.7 Optimal number of clusters

3.7.1 Elbow plot (k-means within-cluster sum of squares)

The k-means WSS elbow method is applied to the Harmony embedding (30 PCs, $k = 2-40$) to identify a natural inflection point in the WSS curve (Figure 4):

```
pca_data <- Embeddings(pbmc_harmony, "pca")[, 1:30]
wss <- sapply(2:40, function(k) kmeans(pca_data, centers = k, nstart = 10)$tot.withinss)
```

3.7.2 Clustree resolution sweep

The Leiden algorithm (algorithm = 4) is applied across ten resolutions (0.05–1.0) using the Harmony embedding. The `clustree` package visualises the stability of cluster partitions as resolution increases (Figure 5):

```
for (res in c(0.05, 0.15, 0.30, 0.40, 0.50, 0.60, 0.7, 0.8, 0.9, 1.0)) {
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)
}
clustree(clu, prefix = "RNA_snn_res.")
```

A resolution of **0.3** was selected, yielding a biologically interpretable partition consistent with known *Arabidopsis* root cell types.

3.8 Final clustering

The Leiden algorithm at resolution 0.3 with $k = 30$ neighbours on the Harmony embedding produces the final cluster assignment:

```
pbmc_harmony <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = 1:30, k.param = 30) %>%
  FindClusters(resolution = 0.3, algorithm = 4, verbose = FALSE)
```

Post-integration UMAPs are saved coloured by `orig.ident` (Figure 6) and by `seurat_clusters` (Figure 7). Bar plots of cell counts per cluster per sample are produced with `plot_integrated_clusters` (Figure 8).

3.9 Pseudobulk generation and replicate correlation

Single-cell counts are aggregated per sample (preserving biological replicates) and per condition (replicates summed) using `generate_pseudobulk`. The function handles the multi-layer sparse matrix structure of Seurat 5 merged objects:

```
pseudobulk <- generate_pseudobulk(pbmc_harmony, group_by = "orig.ident")
pseudobulk$by_sample # matrix: genes x samples
pseudobulk$by_condition # matrix: genes x conditions
```

Replicate correlation is assessed with `plot_replicate_correlation` to confirm reproducibility between R1 and R2 replicates within each nitrogen condition (Figure 9).

3.10 Marker gene identification

Positive marker genes for each cluster are identified using the Wilcoxon rank-sum test (minimum expressed fraction 25%, log2 FC threshold 0.25). Results are cached on disk to avoid recomputation:

```
markers <- find_markers(  
  pbmc_harmony,  
  output_file = "metodologia/FindAllMarkers_0.3C.tsv"  
)
```

3.11 Cell-type annotation

3.11.1 Marker gene reference

A curated reference file (`biblio_marks.txt`) containing known *Arabidopsis* root cell-type marker genes was cross-referenced with the FindAllMarkers output. The best-matching cell-type label (by adjusted p-value) was assigned to each cluster:

```
pbmc_harmony <- annotate_by_markers(  
  pbmc_harmony,  
  markers,  
  reference_file = "metodologia/biblio_marks.txt"  
)
```

3.11.2 Label transfer from published atlas

To obtain an independent, unbiased annotation, cell-type labels were transferred from the published *Arabidopsis* root single-cell atlas (GSE273033, Seurat object `GSE273033_seuratObj_for_publication.rds`) using Seurat's anchor-based label transfer workflow:

```
esp <- readRDS("metodologia/GSE273033_seuratObj_for_publication.rds")  
pbmc_harmony <- annotate_by_reference(  
  pbmc_harmony,  
  reference_obj = esp,  
  reference_col = "annotation"  
)
```

`FindTransferAnchors` identifies shared cell states in PCA space across the query and reference datasets; `TransferData` then projects the reference labels onto query cells. The resulting `celltype_reference` metadata column provides a second, atlas-validated annotation for each cell.

3.12 Export for Python-based analyses

The annotated Seurat object is exported to AnnData h5ad format for downstream trajectory analysis with `scFates` and `palantir`:

```
exportar_para_scanpy(  
  pbmc_harmony,  
  outfile = "results/pbmc_harmony_final.h5ad",
```

```
use_reduc = c("pca", "umap", "harmony")
)
```

4 Notes

1. **Arabidopsis-specific organellar prefixes** — Unlike mammalian datasets where mitochondrial genes begin with MT-, in *Arabidopsis* the prefixes are **ATMG** (mitochondrial) and **ATCG** (chloroplast/plastid). The 5% chloroplast threshold is less stringent than the mitochondrial threshold because some root cell types with active chloroplast-to-leucoplast conversion may show elevated `percent.cp` for biological rather than technical reasons.
 2. **CellBender parameter defaults** — Default CellBender parameters (learning rate, total droplets, expected cells) were used for all samples. For datasets with unusual cell size distributions or very high/low cell recovery, these should be tuned per sample.
 3. **Harmony vs. RPCA** — Harmony was chosen over Seurat’s RPCA integration because it operates directly in PCA space without requiring a common feature space across samples, and it scales efficiently to five samples. For more severe batch effects or cross-species comparisons, scVI (accessible via SeuratWrappers) may be preferable.
 4. **Resolution selection at 0.3** — The choice of resolution 0.3 was informed by the convergence of two independent metrics: the k-means elbow suggesting ~10–12 major transcriptional programmes, and the clustree plateau indicating that cluster identities stabilise below resolution 0.4. This resolution recovers cell types consistent with canonical *Arabidopsis* root anatomy (epidermis, cortex, endodermis, pericycle, stele, quiescent centre, columella).
 5. **Pseudobulk vs. single-cell differential expression** — All condition-level differential expression analyses use pseudobulk counts aggregated per sample, not per cell. This approach respects the statistical independence of biological replicates and avoids the inflated degrees of freedom that arise from treating individual cells as independent observations.
 6. **Reference atlas** — GSE273033 is the Shahan et al. (2022) *Arabidopsis* root cell type atlas, which provides a high-confidence annotation framework covering all major root cell identities. Label transfer scores should be inspected to identify low-confidence assignments, which may represent novel or transitional cell states not present in the reference.
-

References

1. Hao Y, *et al.* (2024). Dictionary learning for integrative, multimodal and scalable single-cell analysis. *Nature Biotechnology* **42**, 293–304.
2. Korsunsky I, *et al.* (2019). Fast, sensitive and accurate integration of single-cell data with Harmony. *Nature Methods* **16**, 1289–1296.
3. McGinnis CS, Murrow LM, Gartner ZJ (2019). DoubletFinder: doublet detection in single-cell RNA sequencing data using artificial nearest neighbors. *Cell Systems* **8**, 329–337.

4. Fleming SJ, *et al.* (2023). Unsupervised removal of systematic background noise from droplet-based single-cell experiments using CellBender. *Nature Methods* **20**, 1323–1335.
5. Shahan R, *et al.* (2022). A single-cell Arabidopsis root atlas reveals developmental trajectories in wild-type and cell identity mutants. *Developmental Cell* **57**, 543–560.
6. Love MI, Huber W, Anders S (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology* **15**, 550.
7. Zappia L, Oshlack A (2018). Clustering trees: a visualisation for evaluating clusterings at multiple resolutions. *GigaScience* **7**, giy083.
8. Wolf FA, Angerer P, Theis FJ (2018). SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology* **19**, 15.